University College London

Department of Computer Science

A thesis submitted for the degree of Master of Science in Computational Finance

Market Regime Detection from Dynamic Financial Networks

Author
Abbinav Sankar Kailasam

Academic Supervisor
Prof. Fabio Caccioli
University College London

Industrial Supervisor
Dr. Blaz Zlicar
ADIA Lab

August 28, 2026

Declaration

This dissertation is submitted as partial requirement for the MSc Computational Finance degree at UCL. It is substantially the result of my own work except where explicitly indicated in the text.
The dissertation will be distributed to the internal and external examiners, but thereafter may not be copied or distributed except with permission from the author.

Acknowledgements

Foremost, I would like to thank my academic supervisor, Prof. Fabio Caccioli of the UCL Department of Computer Science, for his patient guidance throughout the design and development of the project. I would also like to thank my industry supervisor, Dr. Blaz Zlicar of ADIA Lab, for his feedback on the methodology and its practical relevance to risk management in finance.

Abstract

The dissertation investigates whether the evolving topology of financial correlation networks can be used to detect economically interpretable market regimes. The analysis uses daily returns from four equity markets, treating the S&P 500 as the primary market and the Nikkei 225, FTSE 350, and CSI 300 as cross-market robustness tests. Rolling correlation matrices are computed at three time scales and filtered into sparse network representations. Each network window is embedded using Graph2Vec, and the resulting vectors are standardised, reduced using principal component analysis, and clustered into three regimes: Calm, Transitional, and Crisis. The pipeline is calibrated on the S&P 500 using windows ending before 2020 and validated out of sample against the CBOE Volatility Index (VIX). The calibrated pipeline is then applied to the remaining indices to assess whether the detected regime structure generalises across equity markets. To examine the contribution of network topology, the threshold network is compared with two augmented constructions that add either a minimum spanning tree (MST) or a triangulated maximally filtered graph (TMFG) backbone. A benchmark comparison tests whether the learned Graph2Vec embeddings provide better regime detection than deterministic descriptors based on spectral features and eigenvector centrality. Finally, a portfolio experiment compares an equally weighted buy-and-hold strategy with a regime-timed strategy using the same equity universe to assess the practical value of the detected regimes.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Chapter 2

Literature Review

Chapter 3

Methodology

3.1 Data description

3.1.1 Equity price data

This study covers four equity markets: the S&P 500 in the United States of America (USA), the Nikkei 225 in Japan (JPN), the FTSE 350 in the United Kingdom (UK), and the CSI 300 in China (CHN). In each market, daily adjusted closing prices for the constituent stocks were downloaded from Yahoo Finance for the period from 1 January 2001 to 15 July 2026.

The raw price panels were systematically screened to maintain consistency across the sample and ensure reliable estimates throughout the regime detection pipeline. Stocks with non-positive prices, missing observations, or absolute daily log returns exceeding 0.8 were excluded. Stocks with absolute log returns below 10^{-12} for at least 63 consecutive trading days were also removed, as such flat periods would produce undefined correlations within the shortest rolling window. Before applying this final criterion, dates on which more than 98% of the surviving stocks were unchanged were classified as market-wide non-trading days and removed.

Table 3.1: Initial and final constituent counts, with removals under each screening criterion.

Market	Initial N	Non-positive price	Return jump	Missing data	Flat run	Final N
S&P 500	503	0	5	149	0	349
Nikkei 225	225	1	5	42	0	177
FTSE 350	350	0	27	157	7	159
CSI 300	300	0	8	223	33	36

Table 3.1 reports the number of stocks removed under each screening criterion. The resulting panels contain 349 S&P 500, 177 Nikkei 225, 159 FTSE 350, and 36 CSI 300 stocks. The CSI 300 had the largest reduction, as many of its current constituents do not have complete price histories extending to 2001. The market-wide filter removed no dates

from the S&P 500, 114 from the Nikkei 225, 12 from the FTSE 350, and 104 from the CSI 300.

The constituent lists reflect index membership at the time of data collection. Stocks removed from an index or delisted before the lists were collected are therefore absent from the full sample. This introduces survivorship bias, which is further increased by requiring stocks to have complete price histories.

3.1.2 Volatility index

Daily closing values of the CBOE Volatility Index (VIX) were downloaded from Yahoo Finance for the same period and aligned with the S&P 500 trading dates. The VIX measures the annualised 30-day implied volatility by S&P 500 index options and is commonly used as an indicator of market uncertainty. In this study, it is used to validate the detected S&P 500 regimes out of sample. The VIX is not used as input to the correlation, network construction, or clustering stages of the regime detection pipeline.

3.2 Correlation and distance matrices

The adjusted closing price series for each stock is transformed into daily logarithmic returns to reduce the non-stationarity of price levels and ensure comparability across stocks. For stock i, the return on trading day τ is defined as

$$r_i(\tau) = \ln P_i(\tau) - \ln P_i(\tau - 1) = \ln\left(\frac{P_i(\tau)}{P_i(\tau - 1)}\right), \tag{3.1}$$

where $P_i(\tau)$ denotes the adjusted closing price of stock i on day τ.

To capture changes in the dependence structure of stocks over time, the return series are divided into overlapping windows. The analysis uses the window and step-size pairs

$$(T, \delta T) \in \{(63, 5), (132, 10), (378, 22)\}, \tag{3.2}$$

where T is the number of trading days in each window and δT is the number of days by which the window is advanced. These settings represent short, medium, and long-term market dynamics, with windows of approximately three, six, and eighteen months advanced at weekly, fortnightly, and monthly intervals, respectively.

Within each window t, the equal-time Pearson correlation between stocks i and j is computed as

$$\rho_{ij}^t = \frac{\langle r_i r_j \rangle_t - \langle r_i \rangle_t \langle r_j \rangle_t}{\sigma_i^t \sigma_j^t}, \tag{3.3}$$

where $\langle \cdot \rangle_t$ denotes the time average over the T observations in window t, and σ_i^t is the corresponding standard deviation of the returns of stock i. The resulting correlation matrix $\mathbf{C}^t = [\rho_{ij}^t]$ is symmetric, has unit diagonal entries, and satisfies $\rho_{ij}^t \in [-1, 1]$.

Each correlation matrix is transformed into a distance matrix using the correlation-based metric introduced by Mantegna (1999),

$$d_{ij}^t = \sqrt{2\left(1 - \rho_{ij}^t\right)}. \tag{3.4}$$

The resulting distances satisfy $d_{ij}^t \in [0, 2]$, with highly correlated stocks placed closer together. The sequences of correlation matrices $\mathbf{C}^t$ and distance matrices $\mathbf{D}^t = [d_{ij}^t]$ provide the inputs to the network construction procedures described in the following section.

3.3 Network construction

For each window t, the correlation matrix $\mathbf{C}^t$ defines a complete weighted graph with N vertices and $N(N-1)/2$ edges. The topology of this graph is identical across windows, with temporal variation appearing only in the edge weights. Since this study investigates regimes through changes in network topology, the correlation matrices are transformed into sparse networks whose edge structures evolve over time. Three undirected and unweighted representations are constructed, a threshold network and its augmentation with either a minimum spanning tree (MST) or a triangulated maximally filtered graph (TMFG) backbone.

The threshold network forms the common base layer. It retains an edge when the corresponding correlation is at least $\theta = 0.65$, giving the edge set

$$E_{\mathrm{T}}^t = \left\{(i, j) : i < j,\ \rho_{ij}^t \geq \theta\right\}, \qquad \theta = 0.65. \tag{3.5}$$

The threshold value was calibrated using the pooled off-diagonal correlations of the S&P 500 but was not recalibrated separately for the other markets. Changes in correlation strength are translated into the addition or removal of edges, allowing network density, connectivity, and local neighbourhood structure to vary between windows. However, the threshold network does not guarantee global connectivity, so some stocks may form separate components or remain isolated.

The first alternative backbone is the MST, constructed from the distance matrix $\mathbf{D}^t$ using Kruskal's algorithm (Mantegna, 1999; Onnela et al., 2003). The complete procedure is presented in Algorithm A1 in Appendix A. The MST connects all N stocks with exactly $N-1$ edges while minimising the total edge distance. Some of these edges may correspond to correlations below θ but are retained because they connect otherwise separated parts of the network. The MST–threshold network combines this backbone with the threshold layer,

$$E_{\text{MST-T}}^t = E_{\text{MST}}^t \cup E_{\mathrm{T}}^t. \tag{3.6}$$

This construction preserves the changing density of the threshold network while ensuring global connectivity. However, the MST backbone contains no cycles and retains minimal local structure.

The second alternative backbone is the TMFG. It begins with a four-node clique and sequentially inserts each remaining stock into the triangular face that maximises the correlation gain (Massara et al., 2017). The complete procedure is presented in Algorithm A2 in Appendix A. The resulting graph is connected and maximal planar, with exactly $3N-6$ edges. Unlike the MST, it contains cycles and triangular motifs that retain a richer local topology. The TMFG–threshold network combines this backbone with the threshold layer,

$$E^t_{\text{TMFG-T}} = E^t_{\text{TMFG}} \cup E^t_{\text{T}}. \tag{3.7}$$

This augmented network remains connected due to the TMFG backbone, but may no longer be planar after the addition of threshold edges.

Both augmented networks contain the threshold layer by construction,

$$E^t_{\text{T}} \subseteq E^t_{\text{MST-T}}, \qquad E^t_{\text{T}} \subseteq E^t_{\text{TMFG-T}}. \tag{3.8}$$

The MST–threshold and TMFG–threshold networks are alternative extensions of the same threshold base. Neither augmented network is guaranteed to contain the other since the two backbones may select different edge sets.

This design provides a controlled comparison of the topological information added by each backbone. The threshold network captures changes in density and connectivity, the MST adds the minimum structure required for global connectivity, and the TMFG adds a denser organisation containing cycles and triangular motifs. Comparing the three representations tests whether regime detection is driven by changes in threshold network density or benefits from the additional topology introduced by the two backbones.

3.4 Network and matrix representations

The graph or correlation matrix from each window t must be converted into a fixed-length vector before clustering. Graph2Vec provides the primary learned representation of network topology. Two deterministic benchmarks are also considered, an eigenvector centrality vector derived from the network and a spectral descriptor derived from the correlation matrix. These benchmarks enable the value of learned topological features to be assessed relative to simpler representations.

3.4.1 Graph2Vec

Graph2Vec learns an unsupervised fixed-length representation of an entire graph by treating it as a document and its rooted subgraphs as words (Narayanan et al., 2017). For the unlabelled networks used in this study, each node is initially labelled by its degree. Three Weisfeiler–Lehman (WL) relabelling iterations generate rooted subgraph labels describing progressively broader neighbourhoods around each node (Shervashidze et al., 2011). A distributed bag-of-words Doc2Vec model then learns a graph representation from the distribution of these rooted subgraph features (Le and Mikolov, 2014). Networks with similar local topology are therefore expected to have similar representations.

The model is fitted separately to each network construction for 100 epochs, with an embedding dimension of 128. The vocabulary is built from the training windows, excluding features observed fewer than five times. For the test period, embeddings are inferred using the fitted model and fixed vocabulary, with unseen features omitted. This preserves the train–test separation and prevents information in the test period from entering the learned representation. The complete training and inference procedure is presented in Algorithm A3 in Appendix B.

Across n windows, the resulting vectors form a matrix $\mathbf{\Phi} \in \mathbb{R}^{n\times 128}$, ordered by window end date and passed to the dimensionality reduction and clustering stages. Comparing embeddings across the threshold, MST–threshold, and TMFG–threshold networks tests whether the detected regime structure changes with the additional topology introduced by the alternative backbones.

3.4.2 Eigenvector centrality descriptor

Eigenvector centrality provides a deterministic representation of node importance based on the centrality of neighbouring nodes (Bonacich, 1972). For the binary adjacency matrix $\mathbf{A}^t$, the centrality vector $\mathbf{v}^t$ satisfies

$$\mathbf{A}^t\mathbf{v}^t = \lambda^t_{\max}\mathbf{v}^t, \qquad \|\mathbf{v}^t\|_2 = 1, \tag{3.9}$$

where $\lambda^t_{\max}$ is the largest eigenvalue of $\mathbf{A}^t$. For a connected undirected graph, the associated eigenvector has strictly positive values.

The ordering of stocks is fixed across windows within each market, producing an N-dimensional descriptor for every network. Since disconnected threshold networks may produce centrality concentrated in a single component, the descriptor is restricted to the connected MST–threshold and TMFG–threshold networks. It contains no fitted parameters and provides a deterministic benchmark for whether network topology encoded through node importance can detect market regimes.

3.4.3 Spectral descriptor

The spectral descriptor is computed directly from $\mathbf{C}^t$ and therefore provides a deterministic benchmark without network construction. For T observations and N stocks, the sample correlation matrix has rank at most $\min(N, T-1)$. When $N > T-1$, at least $N-(T-1)$ eigenvalues are zero and are excluded from features describing the positive spectrum.

The Marchenko–Pastur upper edge provides a reference for separating large sample eigenvalues from the random bulk (Laloux et al., 1999; Plerou et al., 1999):

$$\lambda_+ = \left(1 + \sqrt{\frac{1}{Q}}\right)^2, \qquad Q = \frac{T}{N}. \tag{3.10}$$

Let $\lambda^t_1 \geq \cdots \geq \lambda^t_N$ denote the eigenvalues of $\mathbf{C}^t$ and $\mathbf{u}^t_k$ the corresponding normalised eigenvectors.

Nine features are extracted from each window. The mean and standard deviation of the off-diagonal correlations describe overall dependence and its dispersion. The normalised eigenvalue shares λ_k^t/N for $k = 2, 3, 4$ capture subleading spectral structure, while the number of eigenvalues above λ_+ measures the size of the spectrum outside the random bulk. The remaining features are the normalised entropy of the positive bulk eigenvalues and the participation ratios of the first two eigenvectors. The leading eigenvalue is omitted to reduce redundancy with the market-wide dependence represented by the mean correlation.

To quantify the distribution of eigenvalue mass within the bulk, define

$$\mathcal{B}^t = \left\{ i : 0 < \lambda_i^t \leq \lambda_+ \right\}, \qquad m_t = \left| \mathcal{B}^t \right|, \qquad p_i^t = \frac{\lambda_i^t}{\sum_{j \in \mathcal{B}^t} \lambda_j^t}. \tag{3.11}$$

For $m_t > 1$, the normalised spectral entropy is

$$H_{\text{bulk}}^t = -\frac{\sum_{i \in \mathcal{B}^t} p_i^t \ln p_i^t}{\ln m_t}. \tag{3.12}$$

The entropy approaches one when the bulk eigenvalues are distributed evenly and decreases as their mass becomes more concentrated.

For a normalised eigenvector $\mathbf{u}_k^t$, the participation ratio is

$$\mathrm{PR}(\mathbf{u}_k^t) = \frac{1}{\sum_{i=1}^{N} (u_{ik}^t)^4}, \qquad 1 \leq \mathrm{PR}(\mathbf{u}_k^t) \leq N. \tag{3.13}$$

Lower values indicate localisation on a small number of stocks, whereas higher values indicate a more distributed eigenvector.

Together, these features summarise market-wide dependence, variation in pairwise correlations, subleading spectral structure, spectral concentration, and eigenvector localisation. The descriptor therefore tests whether deterministic properties of the correlation matrix contain sufficient information for regime detection without a learned network representation.

3.5 Dimensionality reduction and regime detection

For each window, Graph2Vec produces a 128-dimensional embedding, eigenvector centrality an N-dimensional vector, and the spectral descriptor a nine-dimensional vector. Within each experimental configuration, features are standardised to zero mean and unit variance using parameters estimated from the training data. PCA is then fitted to the standardised training representations, retaining the minimum number of components required to explain 95% of the variance. The fitted scaler and PCA transformation are applied unchanged to the test period.

3.5.1 K-means clustering

K-means partitions the reduced representations into K clusters by minimising the within-cluster sum of squared distances:

$$\sum_{k=1}^{K} \sum_{\mathbf{x} \in S_k} \|\mathbf{x} - \mathbf{c}_k\|_2^2, \tag{3.14}$$

where S_k denotes cluster k and $\mathbf{c}_k$ its centroid. The model is fitted to the training windows using 20 random initialisations. Each test window is then assigned independently to its nearest fitted centroid.

3.5.2 Gaussian hidden Markov model

A Gaussian hidden Markov model (HMM) provides a temporally structured alternative to K-means (Hamilton, 1989). Each reduced representation is treated as an observation generated by one of K latent states, with state-specific Gaussian distributions and diagonal covariance matrices. Transitions between states are governed by a $K \times K$ transition matrix.

The HMM is fitted to the training sequence using 20 random initialisations, retaining the model with the highest training log-likelihood. Test states are decoded causally by applying the Viterbi algorithm to observations available up to each window and retaining the final state of the decoded sequence. This ensures that no assigned state depends on future observations. For transition matrix $\mathbf{P}$, the expected dwell time in state i is

$$\text{dwell} * i = \frac{1}{1 - P_{ii}}. \tag{3.15}$$

Both clustering methods return arbitrary numerical labels. To maintain consistent interpretation across experiments, clusters are ranked by the mean value of $\mu(\mathbf{C}^t)$ over the training windows. The lowest-, intermediate-, and highest-correlation clusters are labelled Calm, Transitional, and Crisis, respectively, and displayed in green, blue, and red. The same mapping is applied to the test-period labels. Mean correlation is therefore used only to name the detected clusters and does not influence cluster membership.

The number of regimes is fixed at $K = 3$ throughout the dissertation. This choice is calibrated once on the S&P 500 using a K-means elbow analysis and an HMM Akaike information criterion comparison, then held fixed for all subsequent experiments.

3.6 Experimental design and validation

3.6.1 Training and test design

The S&P 500 is used as the primary calibration market. The threshold, number of regimes, model settings, and experimental design are selected using this market and then held fixed. Within each market, all data-dependent models are fitted using windows ending on or before 31 December 2019. This includes Graph2Vec where applicable, feature standardisation, PCA, and clustering. Windows ending after this date form the test period and are processed using the corresponding fitted transformations and models.

For the Nikkei 225, FTSE 350, and CSI 300, the calibrated methodology and hyperparameters are retained, while model parameters are estimated separately from each market's pre-2020 data. The cross-market analysis therefore tests whether the methodology calibrated on the S&P 500 generalises across markets, rather than whether a single fitted S&P 500 model transfers directly.

3.6.2 Experimental grid

The experimental grid varies the market, rolling-window length, representation, network construction where applicable, and clustering method. Graph2Vec is evaluated on the threshold, MST–threshold, and TMFG–threshold networks. Eigenvector centrality is restricted to the two connected constructions, while the spectral descriptor is computed directly from the correlation matrix. Both K-means and the Gaussian HMM are applied to every configuration in Table 3.2.

Table 3.2: Experimental configurations before applying the two clustering methods.

Representation	Configuration grid	Total
Graph2Vec	4 markets × 3 window lengths × 3 network constructions	36
Eigenvector centrality	4 markets × 3 window lengths × 2 connected constructions	24
Spectral descriptor	4 markets × 3 window lengths	12

The principal topology comparison holds the correlation windows, representation method, dimensionality reduction, and clustering procedure fixed while varying the network construction. This isolates the effect of the alternative edge structures. The representation comparison instead evaluates Graph2Vec against deterministic network- and matrix-based descriptors under the same train–test procedure.

3.6.3 Out-of-sample VIX validation

External validation is performed for the S&P 500 using the CBOE VIX, which measures expected volatility implied by S&P 500 index options (Whaley, 2000). A test window is classified as elevated volatility when its end-date value satisfies VIX ≥ 30. The detected Crisis regime is compared with this indicator using precision, recall, and the F_1 score:

$$\text{precision} = \frac{TP}{TP+FP}, \qquad \text{recall} = \frac{TP}{TP+FN}, \qquad F_1 = \frac{2\ \text{precision recall}}{\text{precision} + \text{recall}}. \tag{3.16}$$

Here, TP, FP, and FN are computed across post-2019 windows using the VIX value on each window end date. Because the rolling windows overlap, these observations are serially dependent and the metrics are interpreted descriptively.

VIX is not used during model fitting or regime labelling and therefore provides an external check on whether the Crisis regime corresponds to elevated market uncertainty out of sample. No equivalent external validator is used for the other markets, where the analysis instead assesses cross-market robustness.

3.7 Embedding geometry comparison

The topology comparison is supplemented by examining how network construction affects the geometry of the Graph2Vec representations. The analysis uses the S&P 500 across all three window lengths and network constructions, using the raw embeddings before standardisation, PCA, and clustering.

For each construction, embeddings are normalised to unit L_2 norm and pairwise cosine distances are computed between windows. Two-dimensional non-metric multidimensional scaling (MDS) is used to visualise the resulting distance structure, with normalised Kruskal stress measuring the distortion of the projection. Cosine self-similarity matrices provide a complementary view of persistent temporal structure. Applying the same procedure to all three constructions allows differences in embedding geometry to be attributed to the network topology supplied to Graph2Vec.

3.8 Portfolio analysis

A pre-specified backtest evaluates whether the detected regimes provide useful information for controlling equity exposure. The configuration is fixed as the S&P 500, raw correlation matrix, $T = 132$, MST–threshold network, Graph2Vec representation, and K-means clustering. It is selected before evaluating portfolio performance and is not chosen ex post from the experimental grid. Post-2019 regime labels are generated using the fitted Graph2Vec, scaler, PCA, and K-means models without refitting. K-means provides a direct out-of-sample rule because each test window is assigned independently to the fixed training centroids.

Two equally weighted portfolio rules are compared. The fully invested benchmark maintains 100% equity exposure, while the regime-timed strategy allocates 100% in Calm, 50% in Transitional, and 0% in Crisis. A signal from a window ending on day t is implemented from the next trading day, so each position uses only information available at the time. Any uninvested allocation earns the 13-week US Treasury bill yield, converted to a daily simple rate as a proxy for the risk-free return.

The equity sleeve is rebalanced to equal weights at each window-advance date, occurring every ten trading days for $T = 132$, and otherwise drifts with realised constituent returns. Turnover from equal-weight rebalancing and regime-driven exposure changes is recorded separately. Both strategies use the same constant-membership stock universe described in Section 3.1. This controls differences in portfolio composition but does not remove the survivorship bias affecting absolute performance.

Performance is assessed using cumulative return, annualised return and volatility, Sharpe ratio, maximum drawdown, and turnover. Robustness is examined by decomposing performance across individual Crisis episodes and non-Crisis periods, recomputing the headline metrics after excluding the COVID-19 episode, applying a moving-block bootstrap with a block length of 132 trading days, and introducing proportional transaction costs of 0, 5, 10, and 20 basis points. Formal significance tests are not reported because the number of distinct crisis episodes is limited despite the larger number of daily observations.

The regime-timed strategy is considered successful only if it achieves both a higher Sharpe ratio and a smaller maximum drawdown than the fully invested benchmark, with both improvements remaining after the COVID-19 episode is excluded.

Chapter 4

Results

Chapter 5

Conclusions and Future Directions

Bibliography

Bonacich, P. 1972. Factoring and weighting approaches to status scores and clique identification. *Journal of Mathematical Sociology* **2**(1) 113–120. doi:10.1080/0022250X.1972.9989806.

Hamilton, J. D. 1989. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica* **57**(2) 357–384.

Laloux, L., P. Cizeau, J.-P. Bouchaud, M. Potters. 1999. Noise dressing of financial correlation matrices. *Physical Review Letters* **83** 1467–1470. doi:10.1103/PhysRevLett.83.1467.

Le, Q., T. Mikolov. 2014. Distributed representations of sentences and documents. *Proceedings of the 31st International Conference on Machine Learning*, *PMLR*, vol. 32. 1188–1196.

Mantegna, R. N. 1999. Hierarchical structure in financial markets. *The European Physical Journal B* **11**(1) 193–197. doi:10.1007/s100510050929.

Massara, G. P., T. Di Matteo, T. Aste. 2017. Network filtering for big data: Triangulated maximally filtered graph. *Journal of Complex Networks* **5**(2) 161–178. doi:10.1093/comnet/cnw015.

Narayanan, A., M. Chandramohan, R. Venkatesan, L. Chen, Y. Liu, S. Jaiswal. 2017. graph2vec: Learning distributed representations of graphs. *arXiv preprint arXiv:1707.05005* .

Onnela, J.-P., A. Chakraborti, K. Kaski, J. Kertész, A. Kanto. 2003. Dynamics of market correlations: Taxonomy and portfolio analysis. *Physical Review E* **68**(5) 056110. doi:10.1103/PhysRevE.68.056110.

Plerou, V., P. Gopikrishnan, B. Rosenow, L. A. N. Amaral, H. E. Stanley. 1999. Universal and nonuniversal properties of cross correlations in financial time series. *Physical Review Letters* **83**(7) 1471–1474. doi:10.1103/PhysRevLett.83.1471.

Shervashidze, N., P. Schweitzer, E. J. van Leeuwen, K. Mehlhorn, K. M. Borgwardt. 2011. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research* **12** 2539–2561.

Whaley, R. E. 2000. The investor fear gauge. *The Journal of Portfolio Management* **26**(3) 12–17.

Appendix A

Network construction

This appendix presents the algorithms used to construct the MST and TMFG backbones. The algorithms operate on the correlation and distance matrices defined in Section 3.2.

A.1 MST construction

Algorithm A1 constructs the MST from the distance matrix $\mathbf{D}^t$ using Kruskal's algorithm.

Algorithm A1 Kruskal's algorithm for correlation window t

Require: Distance matrix $\mathbf{D}^t$ with vertex set V
Ensure: MST edge set E^t_{MST}

```
E^t_MST ← ∅
ℰ ← {(i, j) : i, j ∈ V, i < j}                                ▷ Candidate edges
Sort ℰ in ascending order of d^t_ij, breaking ties by vertex order
Initialise a separate component for every vertex in V

for all (i, j) ∈ ℰ in sorted order do
    if i and j belong to different components then
        E^t_MST ← E^t_MST ∪ {(i, j)}
        Merge the components containing i and j
        if |E^t_MST| = |V| − 1 then
            break
        end if
    end if
end for
return E^t_MST
```

The procedure terminates with exactly $N - 1$ edges and connects all N stocks.

A.2 TMFG construction

Algorithm A2 presents the TMFG procedure applied to each correlation window, following Massara et al. (2017).

Algorithm A2 TMFG construction for correlation window t

Require: Correlation matrix $\mathbf{C}^t$ with vertex set V

Ensure: TMFG edge set E^t_{TMFG}

1: Set the diagonal of $\mathbf{C}^t$ to zero and denote the resulting matrix by $\mathbf{W}^t$

2: Let $\bar{W}^t$ denote the mean of the entries of $\mathbf{W}^t$

3: For each vertex $i \in V$, compute

$$s^t_i = \sum_{j \in V} W^t_{ij}\, \mathbb{I}\big(W^t_{ij} > \bar{W}^t\big)$$

4: Select the four vertices with the largest s^t_i and denote them by V_0

5: $E^t_{\mathrm{TMFG}} \leftarrow \{(i, j) : i, j \in V_0,\ i < j\}$ ▷ Initial clique K_4

6: $\mathcal{F} \leftarrow \{V_0 \setminus \{v\} : v \in V_0\}$ ▷ Initial triangular faces

7: $U \leftarrow V \setminus V_0$

8: For $v \in U$ and $f = \{a, b, c\} \in \mathcal{F}$, define the correlation gain

$$S^t(v, f) = \rho^t_{va} + \rho^t_{vb} + \rho^t_{vc}$$

9: **while** $U \neq \varnothing$ **do**

10: Select the vertex–face pair with maximum correlation gain:

$$(v^*, f^*) \leftarrow \arg\max_{\substack{v \in U \\ f \in \mathcal{F}}} S^t(v, f)$$

11: Let $f^* = \{a, b, c\}$

12: $E^t_{\mathrm{TMFG}} \leftarrow E^t_{\mathrm{TMFG}} \cup \{(v^*, a), (v^*, b), (v^*, c)\}$

13: Replace f^* by $\{v^*, a, b\}$, $\{v^*, a, c\}$, and $\{v^*, b, c\}$

14: $U \leftarrow U \setminus \{v^*\}$

15: **end while**

16: **return** E^t_{TMFG}

The completed TMFG contains exactly $3N - 6$ edges and is connected and planar by construction.

Appendix B

Network and matrix representations

This appendix presents the Graph2Vec procedure used to represent network topology.

B.1 Graph2Vec embedding

Algorithm A3 summarises the training and out-of-sample inference procedure applied independently to each network (Narayanan et al., 2017).

Algorithm A3 Graph2Vec training and inference

Require: Training graphs $\mathcal{G}_{\text{train}}$ and test graphs $\mathcal{G}_{\text{test}}$
Ensure: One 128-dimensional embedding for each graph

for all $G^t \in \mathcal{G}_{\text{train}}$ **do**
    Label each node by its degree
    Apply three Weisfeiler–Lehman relabelling iterations
    Collect the initial and updated node labels as rooted-subgraph words
    Treat the rooted-subgraph words as the document representing G^t
end for
Construct the vocabulary from all training documents
Discard words occurring fewer than five times
Train the Graph2Vec model for 100 epochs with embedding dimension 128
Store the learned training embeddings in $\mathbf{\Phi}_{\text{train}}$
Freeze the fitted model and training vocabulary

for all $G^t \in \mathcal{G}_{\text{test}}$ **do**
    Extract its rooted-subgraph words using the same procedure
    Discard words absent from the training vocabulary
    Infer a 128-dimensional embedding using the fitted model
end for
Store the inferred test embeddings in $\mathbf{\Phi} * \text{test}$
return $\mathbf{\Phi} * \text{train}, \mathbf{\Phi}_{\text{test}}$

The WL relabelling step in Algorithm A3 is defined as follows. For an unlabelled graph, each node is initially labelled by its degree. At iteration $k+1$, the label of node i is updated using its current label and the labels of its neighbours (Shervashidze et al., 2011),

$$M_i^{(k)} = \mathrm{sort}\left(\left[\ell_j^{(k)} \mid j \in \mathcal{N}(i)\right]\right), \ \ell_i^{(k+1)} \quad = \mathrm{relabel}\left(\ell_i^{(k)} \parallel M_i^{(k)}\right). \tag{B.1}$$

Here, $M_i^{(k)}$ is the sorted list of neighbouring labels, $\parallel$ denotes concatenation, and relabel assigns a common discrete label to identical sequences. Repeating the procedure captures progressively broader neighbourhoods, with the labels from each iteration representing rooted subgraph words.